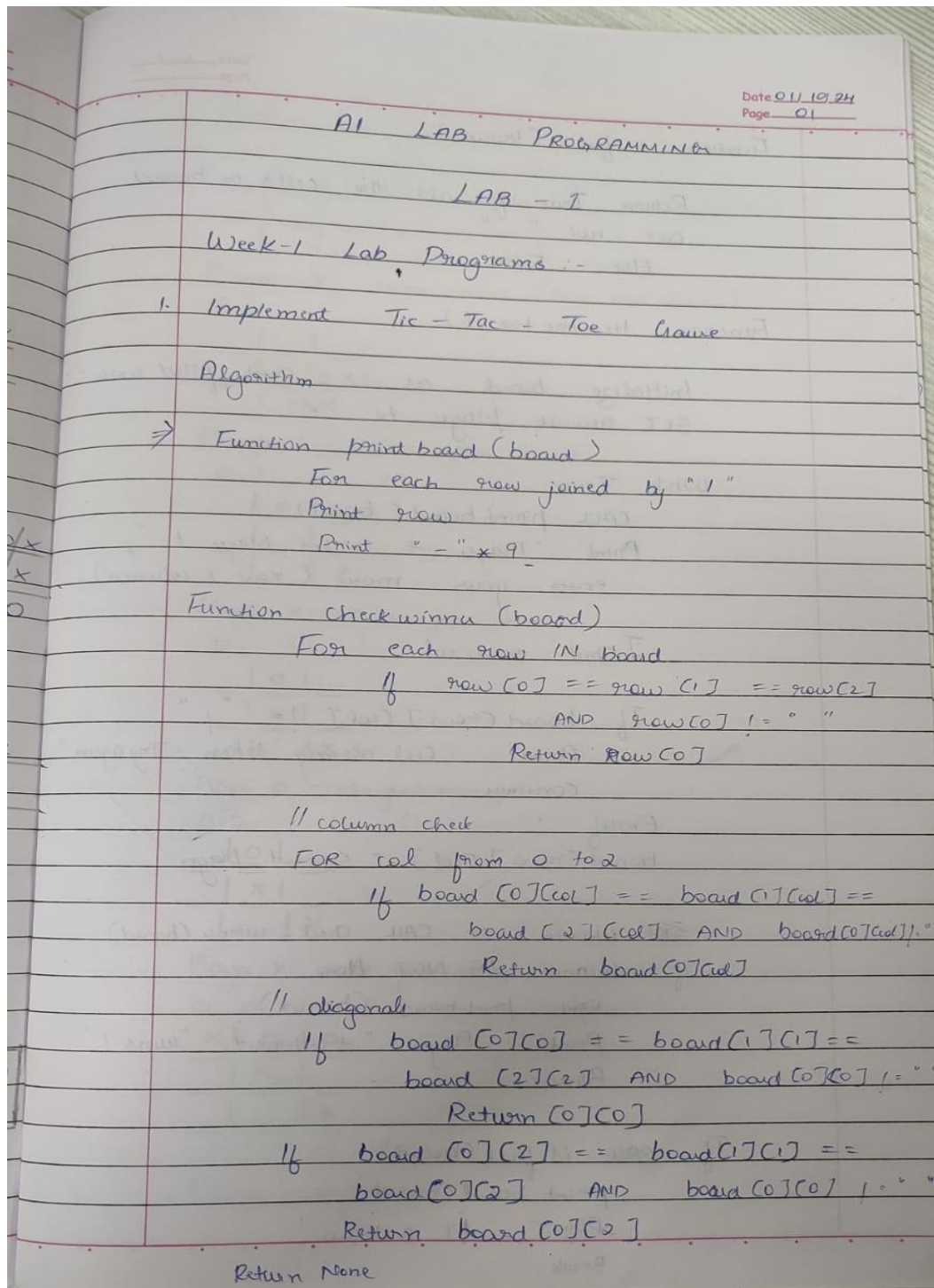


LABORATORY PROGRAM – 1

IMPLEMENT TIC –TAC –TOE GAME

PSEUDOCODE OR ALGORITHM



Date: / /
Page:

Function is full (board)

Return True if all the cells in board
are not " "
Else False

Function tic tac toe ()

Initialize board as 3x3 grid filled with " "
SET current player to "x"

While True

CALL print board (board)

Print "Player + current player + "
enter your move (row & column):"

Input row, col

If board (row) [col] != " "

Print "Cell already taken - Try again"

Continue

Endif

board (row) [col] = current player

~~Set~~ winner = CALL check-winner (board)

If winner IS NOT None

~~Set~~ print board (board)

Print "Player " + current "wins!"

Break

If ~~Set~~ is full (board)

print board (board)

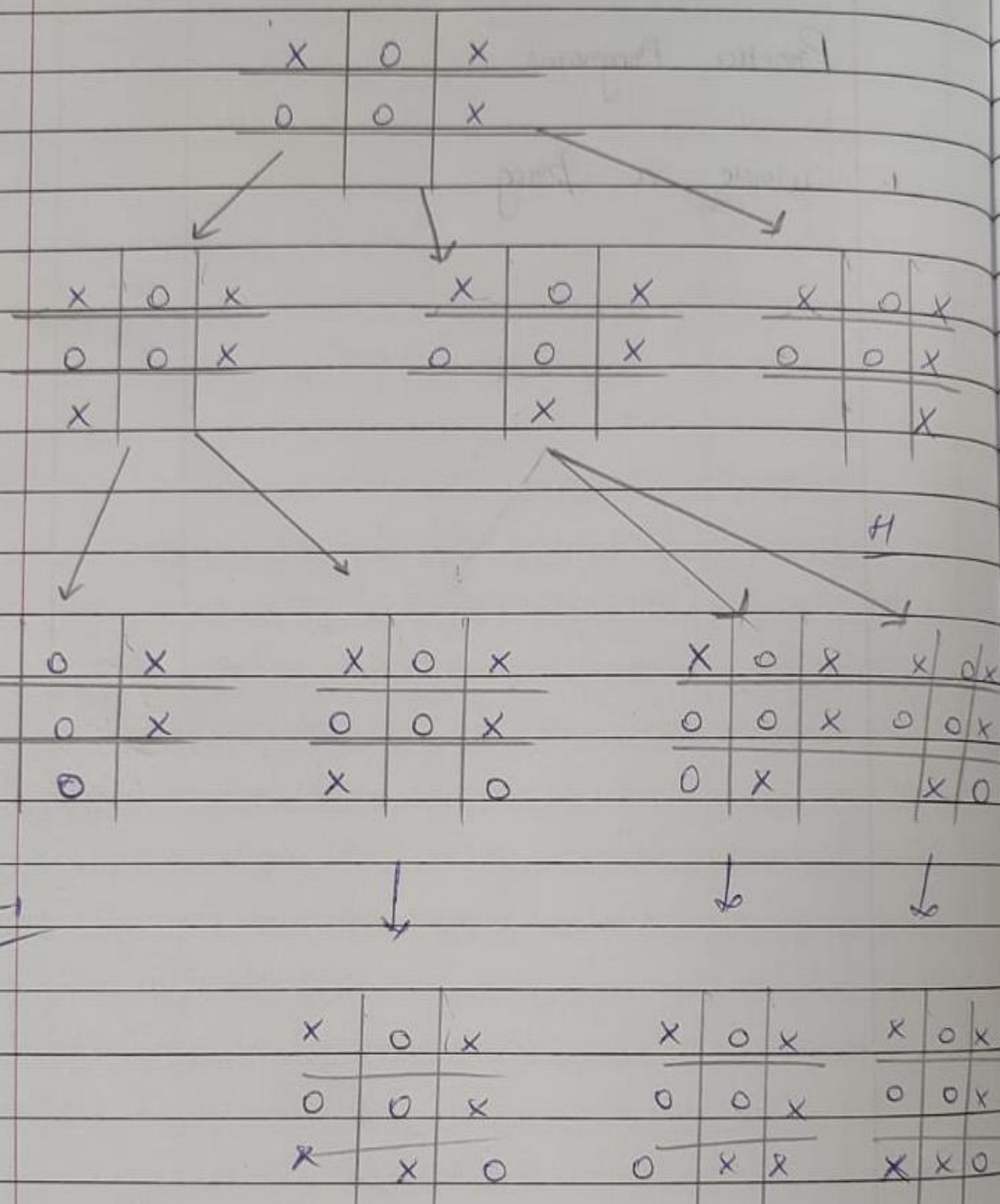
PRINT "It's a draw!"

Break

STATE SPACE TREE

Date / /
Page

State Space Tree for Tic-Tac-Toe



CODE

```
import random

board = ["-", "-", "-",
         "-", "-", "-",
         "-", "-", "-"]

def p_board(player):
    print(board[0] + "|" + board[1] + "|" + board[2])
    print(board[3] + "|" + board[4] + "|" + board[5])
    print(board[6] + "|" + board[7] + "|" + board[8])

def check_win():
    for i in range(0, 7, 3):
        if board[i] == board[i + 1] == board[i + 2] and
board[i] != '-':
            return board[i]

    for i in range(3):
        if board[i] == board[i + 3] == board[i + 6] and
board[i] != '-':
            return board[i]

    if board[0] == board[4] == board[8] and board[0] !=
'-':
        return board[0]
    if board[2] == board[4] == board[6] and board[2] !=
'-':
        return board[2]

    return None

def check_tie():
    if '-' not in board:
        return True
    return False

def play_game():
    current_player = "X"
    game_over = False

    while not game_over:
        p_board(current_player)

        if current_player == "X":
            print("It's your turn (X).")
            try:
                position = int(input("Choose a position from 1-9:
")) - 1
                if position < 0 or position > 8 or board[position]
!= '-':
                    print("Invalid move. Try again.")
                    continue
            except ValueError:
                print("Invalid input. Please enter a number
between 1 and 9.")
                continue
            else:
                print("Computer's turn (O).")
                available_moves = [i for i, spot in
enumerate(board) if spot == '-']
                position = random.choice(available_moves)

        board[position] = current_player
        winner = check_win()

        if winner:
            p_board(current_player)
            print(winner + " won!")
            game_over = True
        elif check_tie():
            p_board(current_player)
            print("It's a tie!")
            game_over = True
        else:
            current_player = "O" if current_player == "X" else
            "X"

    play_game()
```

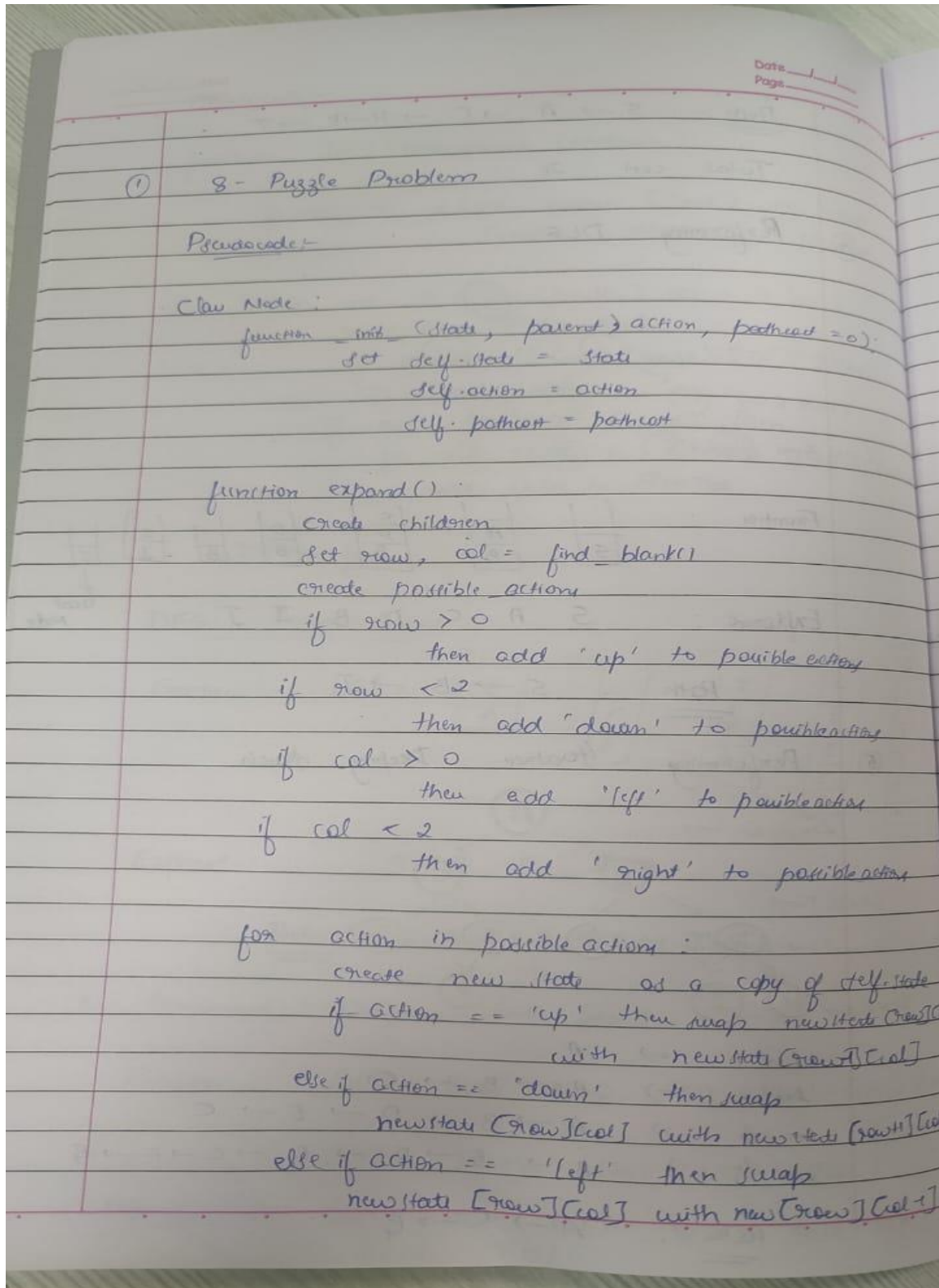
OUTPUT

```
-|-|-  
-|-|-  
-|-|-  
It's your turn (X).  
Choose a position from 1-9: 5  
-|-|-  
-|X|-  
-|-|-  
Computer's turn (O).  
-|O|-  
-|X|-  
-|-|-  
It's your turn (X).  
Choose a position from 1-9: 7  
-|O|-  
-|X|-  
X|-|-  
Computer's turn (O).  
-|O|O  
-|X|-  
X|-|-  
It's your turn (X).  
Choose a position from 1-9: 1  
X|O|O  
-|X|-  
X|-|-  
Computer's turn (O).  
X|O|O  
-|X|O  
X|-|-  
It's your turn (X).  
Choose a position from 1-9: 4  
X|O|O  
X|X|O  
X|-|-  
X won!
```

LABORATORY PROGRAM - 2

SOLVE 8 PUZZLE PROBLEMS

PSEUDOCODE OR ALGORITHM



Date: __/__/__
Page: __

```

else if action == 'right' then
    swap newstate [row][col] with newstate [row][col+1]
append new Node (new_state, self, action, self.path cost + 1) to children
return children

```

```

function find blank():
    for row from 0 to 2
        for col from 0 to 2
            if self.state [row][col] == 0 then
                return row, col

```

```

function depth first search (initial, goal)

```

```

    set frontier = [Node (initial)]
    set explored = empty set
    while frontier is not empty:
        set node = frontier.pop()
        if node.state == goal state then
            return node
        add tuple of node state to explored
        for child in node.expand():
            if tuple of child.state not in explored then append child to frontier
    return none

```

```

set initial_state = [[1, 2, 3], [0, 4, 6], [7, 5, 8]]
set goal_state = [[1, 2, 3], [4, 5, 6], [7, 8, 0]]
set solution = depth first search (initial state, goal state)

```

STATE SPACE TREE

8-Puzzle Problem (Using BFS)

Date: / /
Page:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{bmatrix}$$

 Goal State

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 0 & 6 \\ 7 & 5 & 8 \end{bmatrix}$$

left down right up

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 6 \\ 7 & 5 & 8 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 0 & 8 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 6 & 0 \\ 7 & 5 & 8 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 3 \\ 4 & 2 & 6 \\ 7 & 5 & 8 \end{bmatrix}$$

up down right left

$$\begin{bmatrix} 0 & 2 & 3 \\ 1 & 4 & 6 \\ 7 & 5 & 8 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 7 & 4 & 6 \\ 0 & 5 & 8 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 0 & 7 & 8 \end{bmatrix}$$

Goal State

CODE

```
from collections import deque
import time

goal_state = [[1, 2, 3],
              [4, 5, 6],
              [7, 8, 0]]

def find_zero(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return (i, j)

def move(state, direction):
    new_state = [row[:] for row in state]
    zero_pos = find_zero(state)
    i, j = zero_pos

    if direction == "up" and i > 0:
        new_state[i][j], new_state[i-1][j] = new_state[i-1][j], new_state[i][j]
    elif direction == "down" and i < 2:
        new_state[i][j], new_state[i+1][j] = new_state[i+1][j], new_state[i][j]
    elif direction == "left" and j > 0:
        new_state[i][j], new_state[i][j-1] = new_state[i][j-1], new_state[i][j]
    elif direction == "right" and j < 2:
        new_state[i][j], new_state[i][j+1] = new_state[i][j+1], new_state[i][j]
    else:
        return None

    return new_state

def is_goal(state):
    return state == goal_state

def print_state(state):
    for row in state:
        print(row)
    print("\n")

def dfs(initial_state):
    stack = [(initial_state, [])]
    visited = set()

    while stack:
        state, path = stack.pop()
        print("Exploring state in DFS:")
        print_state(state)

        if is_goal(state):
            return path

        visited.add(str(state))
        for direction in ["up", "down", "left", "right"]:
            new_state = move(state, direction)
            if new_state and str(new_state) not in visited:
                stack.append((new_state, path + [direction]))

    return None

def get_initial_state():
    print("Enter the initial state of the 8-puzzle (0 for empty space):")
    initial_state = []
    for i in range(3):
        row = list(map(int, input(f"Enter row {i+1} (space-separated): ").strip().split()))
        if len(row) != 3:
            raise ValueError("Each row must contain exactly 3 numbers.")
        initial_state.append(row)
    return initial_state

if __name__ == "__main__":
    initial_state = get_initial_state()

    print("Initial State:")
    print_state(initial_state)

    start_time_dfs = time.time()
    print("Solving using DFS:")
    dfs_solution = dfs(initial_state)
    end_time_dfs = time.time()
    if dfs_solution:
        print("DFS Solution:", dfs_solution)
    else:
        print("No solution found with DFS.")
    print(f"Time taken by DFS: {end_time_dfs - start_time_dfs:.6f} seconds")
```

OUTPUT

```
Enter the initial state of the 8-puzzle (0 for empty space):  
Enter row 1 (space-separated): 1 2 3  
Enter row 2 (space-separated): 4 6 5  
Enter row 3 (space-separated): 8 7 0  
Initial State:  
[1, 2, 3]  
[4, 6, 5]  
[8, 7, 0]
```

```
Solving using DFS:  
Exploring state in DFS:  
[1, 2, 3]  
[4, 6, 5]  
[8, 7, 0]
```

```
Exploring state in DFS:  
[1, 2, 3]  
[4, 6, 5]  
[8, 0, 7]
```

```
Exploring state in DFS:  
[1, 2, 3]  
[4, 6, 5]  
[0, 8, 7]
```

```
Exploring state in DFS:  
[1, 2, 3]  
[0, 6, 5]  
[4, 8, 7]
```

```
Exploring state in DFS:  
[1, 2, 3]  
[4, 8, 5]  
[7, 0, 6]
```

```
Exploring state in DFS:  
[1, 2, 3]  
[4, 8, 5]  
[7, 6, 0]
```

```
Exploring state in DFS:  
[1, 2, 3]  
[4, 0, 5]  
[7, 8, 6]
```

```
Exploring state in DFS:  
[1, 2, 3]  
[4, 5, 0]  
[7, 8, 6]
```

```
Exploring state in DFS:  
[1, 2, 3]  
[4, 5, 6]  
[7, 8, 0]
```

```
DFS Solution: ['left', 'left', 'up', 'right', 'right', 'down', 'left', 'le  
Time taken by DFS: 0.183756 seconds
```

LABORATORY PROGRAM – 3

IMPLEMENT ITERATIVE DEEPENING SEARCH ALGORITHM

PSEUDOCODE OR ALGORITHM

② Implement iterative deepening depth first search :-

```
Function iterative_DS (graph, start, goal)

    set depth_limit = 1
    while true:
        print "Depth limit:", depth_limit
        set stack = [(start, [start])]
        while stack is not empty:
            set (node, path) = POP from stack
            print "Visiting:", node, "Current Path:", path
            if node == goal then:
                return path
            if length of path < depth_limit:
                for neighbour IN graph (node):
                    if neighbour NOT in path then:
                        push (neighbour, path + [neighbour])
                        to stack
        depth_limit += 1
```

Example:-

```
graph = {
    'A' : ['B', 'C'],
    'B' : ['D', 'E'],
    'C' : ['F', 'G'],
    'D' : ['H'],
    'E' : ['I'],
    'F' : [],
    'G' : ['J'],
    'J' : []
}
```

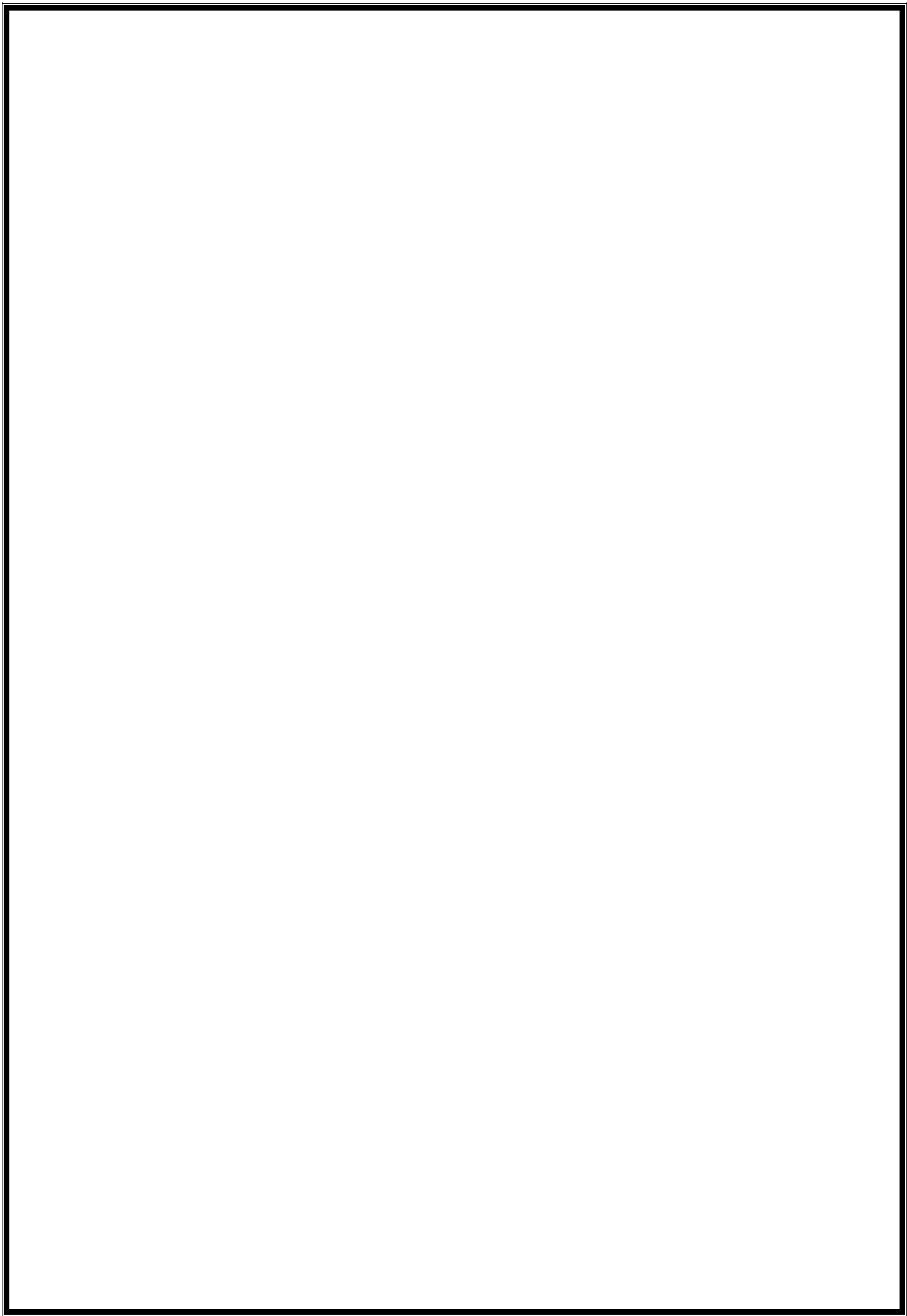
start_node = 'A'
goal_node = 'C'

path = iterative_DS (graph, start_node, goal_node)

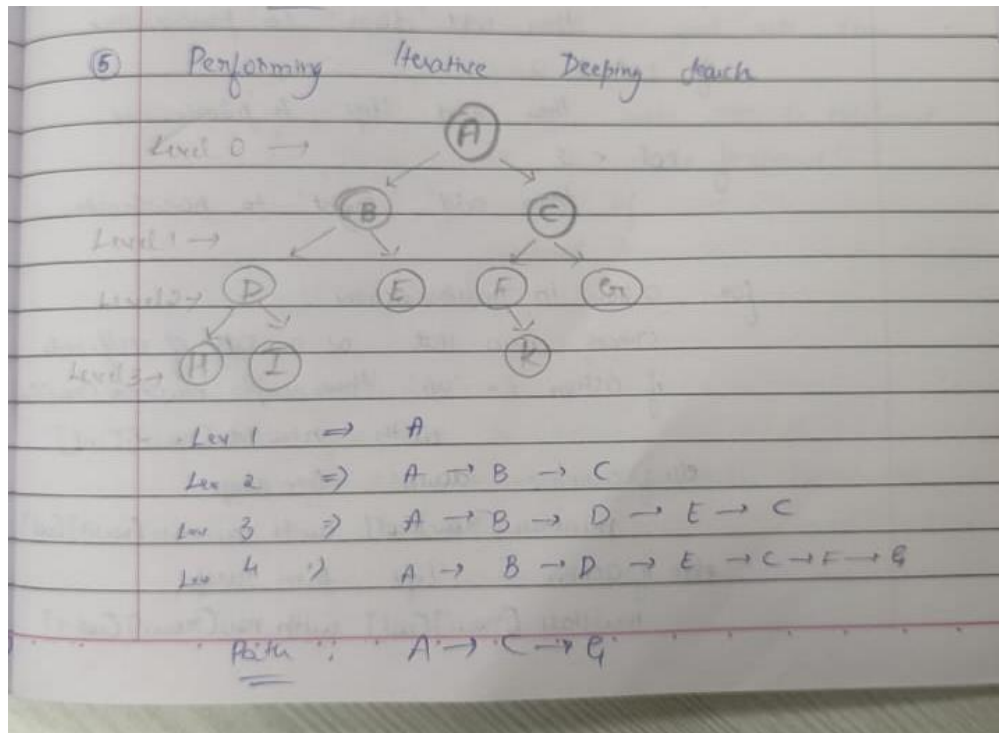
if path is NOT None then :
 print "Path found : ", Path
else
 print "No path found."

⇒ OUTPUT

Path found : ['A', 'C', 'C']



STATE SPACE TREE



CODE

```
class Node:
    def __init__(self, state, parent=None):
        self.state = state
        self.parent = parent

    def path(self):
        node, result = self, []
        while node:
            result.append(node.state)
            node = node.parent
        return result[::-1]

def iterative_deepening_search(problem):
    depth = 0
    while True:
        print(f"Exploring depth: {depth}")
        result, _ = depth_limited_search(problem, depth)
        if result is not None and result != 'cutoff':
            return result
        depth += 1

def depth_limited_search(problem, limit):
    frontier = [Node(problem.initial_state)]
    explored = set()
    cutoff_occurred = False

    while frontier:
        node = frontier.pop()

        if problem.is_goal(node.state):
            return node.path(), explored

        if node.state not in explored:
            explored.add(node.state)
            if len(node.path()) - 1 < limit:
                for child in problem.expand(node.state):
                    frontier.append(Node(child, node))
            else:
                cutoff_occurred = True

    return 'cutoff' if cutoff_occurred else None, explored

class GraphProblem:
    def __init__(self, initial_state, goal_state,
adjacency_list):
        self.initial_state = initial_state
        self.goal_state = goal_state
        self.adjacency_list = adjacency_list

    def is_goal(self, state):
        return state == self.goal_state

    def expand(self, state):
        return [neighbor for neighbor in
self.adjacency_list.get(state, [])]

    def get_graph_from_input():
        adjacency_list = {}
        initial_state = input("Enter the initial state: ").strip()
        goal_state = input("Enter the goal state: ").strip()

        print("Enter the adjacency list for the graph
(neighbors of each node).")
        print("Type 'done' when finished.")

        while True:
            node = input("Enter node (or 'done' to finish):
").strip()
            if node.lower() == 'done':
                break
            neighbors_input = input(f"Enter neighbors of
{node} separated by spaces: ").strip()
            neighbors = neighbors_input.split()
            adjacency_list[node] = [neighbor.strip() for
neighbor in neighbors]

        return GraphProblem(initial_state, goal_state,
adjacency_list)

if __name__ == "__main__":
    problem = get_graph_from_input()
    solution = iterative_deepening_search(problem)

    if solution:
        print("Solution Path:", solution)
    else:
        print("No solution found.")
```

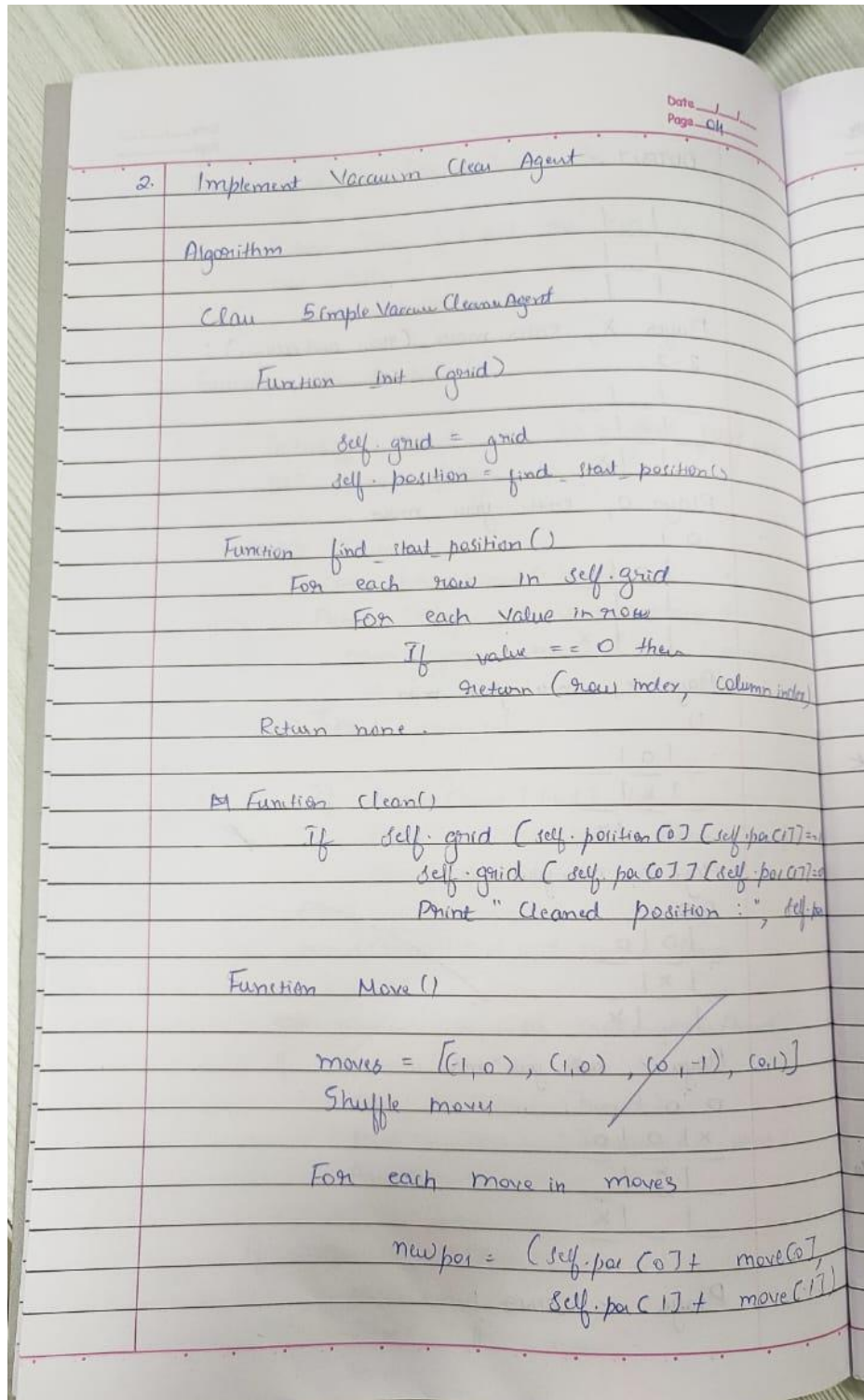
OUTPUT

```
Enter the initial state: A
Enter the goal state: G
Enter the adjacency list for the graph (neighbors of each node).
Type 'done' when finished.
Enter node (or 'done' to finish): A
Enter neighbors of A separated by spaces: B C
Enter node (or 'done' to finish): B
Enter neighbors of B separated by spaces: D E
Enter node (or 'done' to finish): D
Enter neighbors of D separated by spaces: H I
Enter node (or 'done' to finish): C
Enter neighbors of C separated by spaces: F G
Enter node (or 'done' to finish): F
Enter neighbors of F separated by spaces: K
Enter node (or 'done' to finish): done
Exploring depth: 0
Exploring depth: 1
Exploring depth: 2
Solution Path: ['A', 'C', 'G']
```

LABORATORY PROGRAM – 4

IMPLEMENT VACUUM CLEANER AGENT

PSEUDOCODE OR ALGORITHM



Date: / /
Page:

```

if is_within_bounds(new_pos)
    self.pos = new_pos
    Print "Moved to pos ", self.pos
    Break

Function is_within_bounds(position)
    Return (0 <= position[0] < Length(self.grid)
            AND (0 <= position[1] < Length(self.grid))

Function run()
    While True
        Call clean()
        move()

if ! (1 in rows for row in grid) Then
    Print "All positions cleaned!"
    Break

grid = [(0,1)]

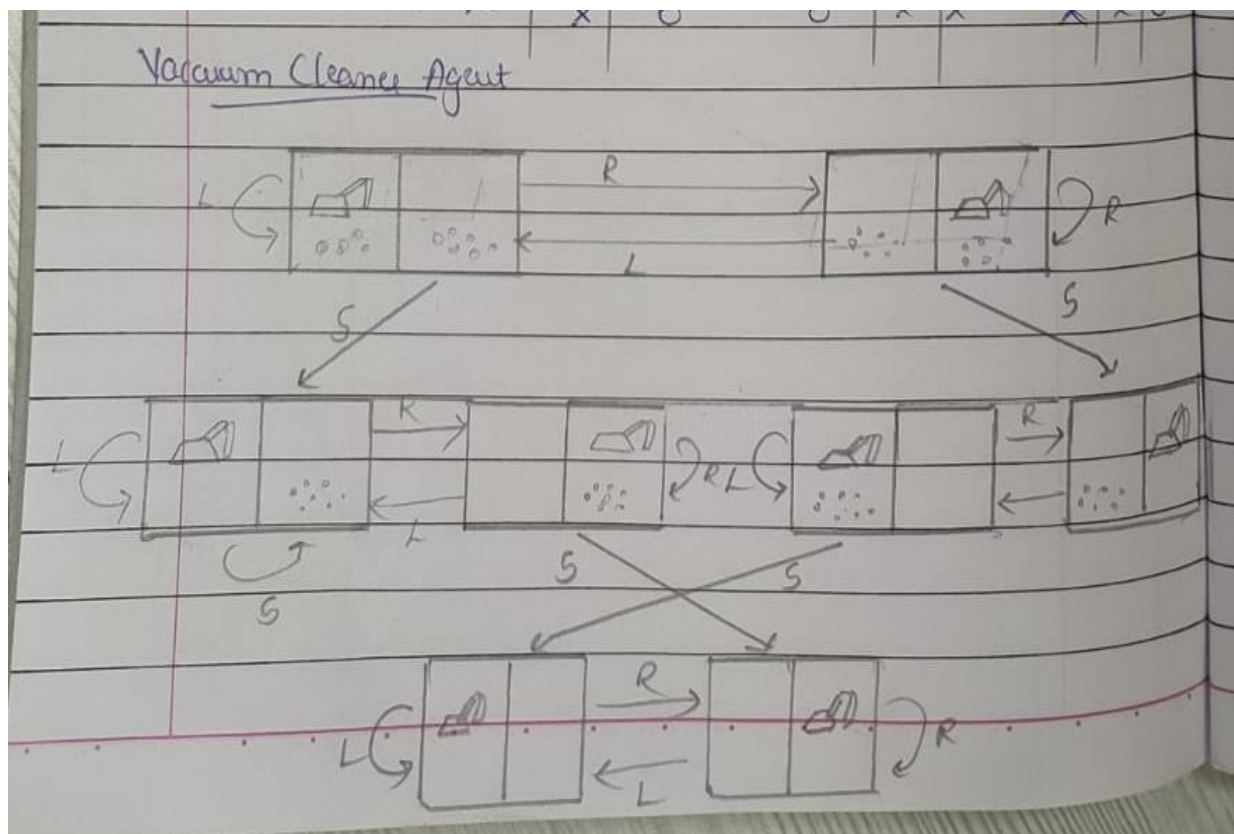
Output:
Moved to position: (0,1)
Cleaned position: (0,1)
Moved to position: (0,0)

All positions cleaned!

```

1/10

STATE SPACE TREE



CODE

```
def vacuum_world():
    goal_state = {'A': '0', 'B': '0'}
    cost = 0
    location_input = input("Enter the initial location of
the vacuum cleaner (A or B): ")
    status_input = input(f"Enter the status of room
{location_input} (0 for Clean, 1 for Dirty): ")
    status_input_complement = input(f"Enter the status
of the other room ({'B' if location_input == 'A' else
'A'}) (0 for Clean, 1 for Dirty): ")

    # Set the initial state based on user input
    initial_state = {
        'A': status_input if location_input == 'A' else
status_input_complement,
        'B': status_input_complement if location_input ==
'A' else status_input
    }

    print("\nInitial Location Condition:", initial_state)

    if location_input == 'A':
        print("\nVacuum cleaner is placed in room A.")

        if status_input == '1':
            print("Room A is Dirty. Cleaning room A...")
            goal_state['A'] = '0'
            cost += 1
            print("Cost for cleaning room A:", cost)

            if status_input_complement == '1':
                print("\nRoom B is also Dirty. Moving to
room B...")
                cost += 1
                print("Cost for moving to room B:", cost)
                print("Cleaning room B...")
                goal_state['B'] = '0'
                cost += 1
                print("Cost for cleaning room B:", cost)
            else:
                print("\nRoom B is already Clean. No further
action needed.")
        else:
            print("Room A is already Clean.")

        if status_input_complement == '1':
            print("\nRoom B is Dirty. Moving to room
B...")
            cost += 1
            print("Cost for moving to room B:", cost)

        print("\nCleaning room B...")
        goal_state['B'] = '0'
        cost += 1
        print("Cost for cleaning room B:", cost)

    print("Cleaning room B...")
    goal_state['B'] = '0'
    cost += 1
    print("Cost for cleaning room B:", cost)
    else:
        print("\nRoom B is already Clean. No further
action needed.")

    else:
        print("\nVacuum cleaner is placed in room B.")

        if status_input == '1':
            print("Room B is Dirty. Cleaning room B...")
            goal_state['B'] = '0'
            cost += 1
            print("Cost for cleaning room B:", cost)

            if status_input_complement == '1':
                print("\nRoom A is also Dirty. Moving to
room A...")
                cost += 1
                print("Cost for moving to room A:", cost)
                print("Cleaning room A...")
                goal_state['A'] = '0'
                cost += 1
                print("Cost for cleaning room A:", cost)
            else:
                print("\nRoom A is already Clean. No further
action needed.")
        else:
            print("Room B is already Clean.")

        if status_input_complement == '1':
            print("\nRoom A is Dirty. Moving to room
A...")
            cost += 1
            print("Cost for moving to room A:", cost)
            print("Cleaning room A...")
            goal_state['A'] = '0'
            cost += 1
            print("Cost for cleaning room A:", cost)
        else:
            print("\nRoom A is already Clean. No further
action needed.")

    print("\nGOAL STATE:", goal_state)
    print("Total cost for cleaning:", cost)

vacuum_world()
```

OUTPUT

```
Enter the initial location of the vacuum cleaner (A or B): A
Enter the status of room A (0 for Clean, 1 for Dirty): 1
Enter the status of the other room (B) (0 for Clean, 1 for Dirty): 1
```

```
Initial Location Condition: {'A': '1', 'B': '1'}
```

```
Vacuum cleaner is placed in room A.
Room A is Dirty. Cleaning room A...
Cost for cleaning room A: 1
```

```
Room B is also Dirty. Moving to room B...
Cost for moving to room B: 2
Cleaning room B...
Cost for cleaning room B: 3
```

```
GOAL STATE: {'A': '0', 'B': '0'}
Total cost for cleaning: 3
```